

Stored Procedures and Functions in PostgreSQL

Table of Contents

- [1. Learning Objectives](#)
 - [2. Functions vs Procedures Overview](#)
 - [3. PL/pgSQL Fundamentals](#)
 - [4. Creating Functions](#)
 - [5. Creating Procedures](#)
 - [6. DO Blocks](#)
 - [7. Control Flow and Error Handling](#)
 - [8. Cursors and Batch Processing](#)
 - [9. Function Volatility and Security](#)
 - [10. Returning Sets and Tables](#)
 - [11. Real-World Function Patterns](#)
 - [12. Production Monitoring Queries](#)
 - [13. Common Mistakes](#)
 - [14. Best Practices](#)
 - [15. Performance Considerations](#)
 - [16. Interview Questions & Answers](#)
 - [17. Exercises and Solutions](#)
 - [18. Cross-References](#)
-

Learning Objectives

By the end of this module you will be able to:

- Explain the difference between functions and procedures and when to use each
 - Write PL/pgSQL with proper error handling, variables, and control flow
 - Implement common patterns: audit logging, batch processing, data validation
 - Use function volatility correctly to enable query optimization
 - Return sets of records from functions efficiently
 - Profile and optimize expensive stored routines
-

Functions vs Procedures Overview

Aspect	Function	Procedure
Return value	Must return a value (or VOID)	Cannot return a value
Transaction control	Cannot COMMIT/ROLLBACK	Can COMMIT/ROLLBACK
Called with	SELECT or in expressions	CALL statement
Used in SQL	Yes (in SELECT, WHERE, etc.)	No
Added in	PostgreSQL 1.0	PostgreSQL 11
OUT parameters	Yes	Yes (return via OUT params)

```
-- Function: used in SELECT expressions
SELECT calculate_tax(order_total) FROM orders;

-- Procedure: called with CALL, can manage transactions
CALL process_payroll_batch(2024, 1);
```

PL/pgSQL Fundamentals

```
-- Anatomy of a PL/pgSQL block
DO $$
DECLARE
    -- Variable declarations
    v_counter    INT      := 0;
    v_name       TEXT;
    v_amount     NUMERIC := 0.0;
    v_arr        INT[]    := ARRAY[1,2,3];
    v_rec        RECORD;
    v_row        orders%ROWTYPE;    -- same structure as a table row
    v_id         orders.id%TYPE;     -- same type as a column
    v_found      BOOLEAN;
BEGIN
    -- Assignments
    v_counter := v_counter + 1;
    v_name    := 'Alice';
    SELECT id INTO v_id FROM orders WHERE status = 'pending' LIMIT 1;

    -- Check if SELECT found a row
    IF NOT FOUND THEN
        RAISE NOTICE 'No pending orders found';
    END IF;

    -- String formatting
    RAISE NOTICE 'Counter: %, Name: %', v_counter, v_name;

END;
$;
```

Creating Functions

Basic function

```
-- Scalar function: takes input, returns scalar
CREATE OR REPLACE FUNCTION calculate_discount(
    p_total    NUMERIC,
    p_tier     TEXT DEFAULT 'standard'
)
RETURNS NUMERIC
```

```

LANGUAGE plpgsql
IMMUTABLE -- no DB access, same result for same inputs
AS $$
BEGIN
    RETURN CASE p_tier
        WHEN 'gold' THEN p_total * 0.85 -- 15% discount
        WHEN 'silver' THEN p_total * 0.90 -- 10% discount
        WHEN 'standard' THEN p_total * 0.95 -- 5% discount
        ELSE
            p_total
        END;
END;
$$;

-- Usage
SELECT calculate_discount(100.00, 'gold'); -- 85.00
SELECT id, total, calculate_discount(total, tier) AS discounted
FROM orders o
JOIN customers c ON c.id = o.customer_id;

```

Function with multiple OUT parameters

```

CREATE OR REPLACE FUNCTION get_order_summary(
    p_customer_id INT,
    OUT p_order_count INT,
    OUT p_total_spent NUMERIC,
    OUT p_last_order TIMESTAMPTZ
)
LANGUAGE plpgsql STABLE AS $$
BEGIN
    SELECT
        COUNT(*),
        SUM(total),
        MAX(created_at)
    INTO p_order_count, p_total_spent, p_last_order
    FROM orders
    WHERE customer_id = p_customer_id
        AND status = 'completed';
END;
$$;

-- Usage (OUT parameters returned as a row)
SELECT * FROM get_order_summary(42);
-- p_order_count | p_total_spent | p_last_order
-- 5             | 1234.56       | 2024-01-15 10:00:00+00

```

SQL function (simpler, often faster)

```

-- SQL functions are inlined by the planner (no function call overhead)
CREATE OR REPLACE FUNCTION full_name(first_name TEXT, last_name TEXT)

```

```

RETURNS TEXT
LANGUAGE sql
IMMUTABLE STRICT -- STRICT = returns NULL if any arg is NULL
AS $$
    SELECT first_name || ' ' || last_name;
$$;

-- The planner inlines this – treated as if you wrote first_name || ' ' || last_name
SELECT full_name('Alice', 'Smith');

```

Creating Procedures

```

-- Procedure: can control transactions
CREATE OR REPLACE PROCEDURE process_batch_payments(p_batch_size INT DEFAULT 100)
LANGUAGE plpgsql AS $$
DECLARE
    v_processed INT := 0;
    v_total     INT;
    r           RECORD;
BEGIN
    SELECT COUNT(*) INTO v_total FROM payments WHERE status = 'pending';
    RAISE NOTICE 'Processing % pending payments', v_total;

    FOR r IN
        SELECT id, amount, customer_id
        FROM payments
        WHERE status = 'pending'
        ORDER BY created_at
        LIMIT p_batch_size
    LOOP
        BEGIN
            -- Attempt payment processing
            UPDATE payments SET status = 'processing' WHERE id = r.id;
            -- ... payment gateway API call would go here ...
            UPDATE payments SET status = 'completed' WHERE id = r.id;
            v_processed := v_processed + 1;

        EXCEPTION WHEN OTHERS THEN
            UPDATE payments SET status = 'failed', error_msg = SQLERRM
            WHERE id = r.id;
            RAISE WARNING 'Payment % failed: %', r.id, SQLERRM;
        END;

        -- Commit every 10 payments (procedures can do this; functions cannot)
        IF v_processed % 10 = 0 THEN
            COMMIT;
            RAISE NOTICE 'Committed % payments', v_processed;
        END IF;
    END LOOP;
END;

```

```

        COMMIT;
    RAISE NOTICE 'Done. Processed: %', v_processed;
END;
$$;

-- Call
CALL process_batch_payments(500);

```

DO Blocks

DO executes an anonymous code block — no name, no persistence, great for one-off operations.

```

-- Backfill missing data
DO $$
DECLARE
    v_count INT := 0;
BEGIN
    UPDATE products
    SET slug = lower(regexp_replace(name, '\s+', '-', 'g'))
    WHERE slug IS NULL;

    GET DIAGNOSTICS v_count = ROW_COUNT;
    RAISE NOTICE 'Updated % product slugs', v_count;
END;
$$;

-- Dynamic DDL with DO
DO $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN
        SELECT schemaname, tablename
        FROM pg_tables
        WHERE schemaname = 'public'
            AND tablename LIKE 'temp_%'
    LOOP
        EXECUTE format('DROP TABLE IF EXISTS %I.%I CASCADE',
                        r.schemaname, r.tablename);
        RAISE NOTICE 'Dropped: %.%', r.schemaname, r.tablename;
    END LOOP;
END;
$$;

```

Control Flow and Error Handling

```

CREATE OR REPLACE FUNCTION safe_divide(a NUMERIC, b NUMERIC)
RETURNS NUMERIC LANGUAGE plpgsql IMMUTABLE AS $$

```

```

BEGIN
    IF b = 0 THEN
        RAISE EXCEPTION 'Division by zero: a=%, b=%', a, b
            USING ERRCODE = 'division_by_zero';
    END IF;
    RETURN a / b;
EXCEPTION
    WHEN division_by_zero THEN
        RETURN NULL;
    WHEN OTHERS THEN
        RAISE WARNING 'Unexpected error: %', SQLERRM;
        RETURN NULL;
END;
$$;

-- Full control flow example
CREATE OR REPLACE FUNCTION categorize_order_value(p_total NUMERIC)
RETURNS TEXT LANGUAGE plpgsql IMMUTABLE AS $$
BEGIN
    -- IF-ELSIF-ELSE
    IF p_total IS NULL THEN
        RETURN 'unknown';
    ELSIF p_total < 10 THEN
        RETURN 'micro';
    ELSIF p_total < 100 THEN
        RETURN 'small';
    ELSIF p_total < 1000 THEN
        RETURN 'medium';
    ELSE
        RETURN 'large';
    END IF;
END;
$$;

-- LOOP patterns
CREATE OR REPLACE FUNCTION fibonacci(n INT)
RETURNS INT LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE
    a INT := 0;
    b INT := 1;
    c INT;
    i INT;
BEGIN
    IF n <= 0 THEN RETURN 0; END IF;
    IF n = 1 THEN RETURN 1; END IF;

    FOR i IN 2..n LOOP
        c := a + b;
        a := b;
        b := c;
    END LOOP;
    RETURN b;

```

```

END;
$$;

-- WHILE loop
CREATE OR REPLACE FUNCTION retry_operation(p_max_attempts INT DEFAULT 3)
RETURNS BOOLEAN LANGUAGE plpgsql AS $$
DECLARE
    v_attempt INT := 0;
    v_success BOOLEAN := false;
BEGIN
    WHILE v_attempt < p_max_attempts AND NOT v_success LOOP
        v_attempt := v_attempt + 1;
        BEGIN
            -- Simulated operation
            RAISE NOTICE 'Attempt %', v_attempt;
            -- ... actual work ...
            v_success := true;
        EXCEPTION WHEN OTHERS THEN
            RAISE WARNING 'Attempt % failed: %', v_attempt, SQLERRM;
            PERFORM pg_sleep(0.1 * v_attempt); -- exponential backoff
        END;
    END LOOP;
    RETURN v_success;
END;
$$;

-- CASE statement
CREATE OR REPLACE FUNCTION get_status_label(p_status TEXT)
RETURNS TEXT LANGUAGE plpgsql IMMUTABLE AS $$
BEGIN
    RETURN CASE p_status
        WHEN 'pending' THEN 'Awaiting Processing'
        WHEN 'processing' THEN 'In Progress'
        WHEN 'completed' THEN 'Done'
        WHEN 'failed' THEN 'Error'
        ELSE 'Unknown: ' || p_status
    END;
END;
$$;

```

Cursors and Batch Processing

```

-- Explicit cursor for processing large result sets
CREATE OR REPLACE PROCEDURE reindex_products_batch(p_batch INT DEFAULT 1000)
LANGUAGE plpgsql AS $$
DECLARE
    cur CURSOR FOR
        SELECT id FROM products ORDER BY id;
    r RECORD;
    v_count INT := 0;

```

```

    v_batch_ct INT := 0;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO r;
        EXIT WHEN NOT FOUND;

        UPDATE products
        SET search_rank = calculate_product_rank(r.id)
        WHERE id = r.id;

        v_count      := v_count + 1;
        v_batch_ct := v_batch_ct + 1;

        IF v_batch_ct >= p_batch THEN
            COMMIT;
            v_batch_ct := 0;
            RAISE NOTICE 'Processed % products...', v_count;
        END IF;
    END LOOP;
    CLOSE cur;
    COMMIT;
    RAISE NOTICE 'Done. Total: %', v_count;
END;
$$;

-- Keyset-based batch processing (better for large tables)
CREATE OR REPLACE PROCEDURE backfill_user_tiers(p_batch_size INT DEFAULT 5000)
LANGUAGE plpgsql AS $$
DECLARE
    v_last_id BIGINT := 0;
    v_count   INT;
BEGIN
    LOOP
        WITH updated AS (
            UPDATE users
            SET tier = CASE
                WHEN lifetime_value > 1000 THEN 'gold'
                WHEN lifetime_value > 500  THEN 'silver'
                ELSE 'bronze'
            END
            WHERE id > v_last_id
            AND tier IS NULL
            ORDER BY id
            LIMIT p_batch_size
            RETURNING id
        )
        SELECT COUNT(*), MAX(id) INTO v_count, v_last_id
        FROM updated;

        COMMIT;
        EXIT WHEN v_count < p_batch_size;
    END LOOP;
END;

```



```

        RAISE NOTICE 'Processed up to id %', v_last_id;
        PERFORM pg_sleep(0.01); -- small sleep to reduce lock pressure
    END LOOP;
    RAISE NOTICE 'Backfill complete';
END;
$$;

```

Function Volatility and Security

```

-- Volatility categories (affects query planning and caching):

-- IMMUTABLE: same inputs always produce same output, no DB access
-- Planner can evaluate at plan time, not execution time
CREATE FUNCTION add(a INT, b INT) RETURNS INT
LANGUAGE sql IMMUTABLE AS $$ SELECT a + b $$;

-- STABLE: within a single query, same inputs produce same output
-- Can read DB but not modify it. Suitable for most read functions
CREATE FUNCTION get_tax_rate(region TEXT) RETURNS NUMERIC
LANGUAGE plpgsql STABLE AS $$
BEGIN
    SELECT rate FROM tax_rates WHERE region_code = region INTO STRICT;
    RETURN FOUND; -- Example
END;
$$;

-- VOLATILE (default): can do anything, different result each call
-- pg_random(), now(), INSERT/UPDATE/DELETE, NOTIFY
-- Never inline by planner

-- SECURITY DEFINER: runs with privileges of the function owner
-- Use carefully – SQL injection risk if not parameterized
CREATE FUNCTION admin_get_secret(p_key TEXT)
RETURNS TEXT
LANGUAGE plpgsql
SECURITY DEFINER -- runs as function owner, not caller
SET search_path = public, pg_catalog -- prevent search_path hijacking
AS $$
BEGIN
    RETURN value FROM secrets WHERE key = p_key;
END;
$$;

-- SECURITY INVOKER (default): runs with caller's privileges

```

Returning Sets and Tables

```

-- RETURNS TABLE
CREATE OR REPLACE FUNCTION get_customer_orders(p_customer_id INT)
RETURNS TABLE (
    order_id    BIGINT,
    total       NUMERIC,
    status      TEXT,
    created_at  TIMESTAMPTZ
)
LANGUAGE plpgsql STABLE AS $$
BEGIN
    RETURN QUERY
    SELECT o.id, o.total, o.status, o.created_at
    FROM orders o
    WHERE o.customer_id = p_customer_id
    ORDER BY o.created_at DESC;
END;
$$;

-- Usage (same as a table in FROM clause)
SELECT * FROM get_customer_orders(42);
SELECT * FROM get_customer_orders(42) WHERE status = 'completed';

-- RETURNS SETOF RECORD (dynamic column names)
CREATE OR REPLACE FUNCTION paginate_orders(p_page INT, p_size INT)
RETURNS SETOF orders
LANGUAGE sql STABLE AS $$
    SELECT * FROM orders
    ORDER BY created_at DESC
    LIMIT p_size OFFSET (p_page - 1) * p_size;
$$;

-- RETURNS JSONB (for API endpoints)
CREATE OR REPLACE FUNCTION get_order_json(p_id BIGINT)
RETURNS JSONB LANGUAGE plpgsql STABLE AS $$
DECLARE
    result JSONB;
BEGIN
    SELECT jsonb_build_object(
        'id',      o.id,
        'status',  o.status,
        'total',   o.total,
        'items',   (
            SELECT jsonb_agg(jsonb_build_object(
                'product_id', oi.product_id,
                'quantity',   oi.quantity,
                'price',      oi.unit_price
            ))
            FROM order_items oi WHERE oi.order_id = o.id
        )
    )
    INTO result

```

```

FROM orders o WHERE o.id = p_id;

IF NOT FOUND THEN
    RAISE EXCEPTION 'Order % not found', p_id USING ERRCODE = 'P0002';
END IF;
RETURN result;
END;
$$;

```

Real-World Function Patterns

Audit logging function

```

CREATE TABLE audit_log (
    id          BIGSERIAL PRIMARY KEY,
    table_name TEXT,
    operation   TEXT,
    row_id      BIGINT,
    old_data    JSONB,
    new_data    JSONB,
    user_name   TEXT DEFAULT current_user,
    app_user    TEXT,
    changed_at  TIMESTAMPTZ DEFAULT now()
);

CREATE OR REPLACE FUNCTION audit_trigger_fn()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
    INSERT INTO audit_log (
        table_name, operation, row_id, old_data, new_data,
        app_user
    ) VALUES (
        TG_TABLE_NAME,
        TG_OP,
        COALESCE((NEW.id)::bigint, (OLD.id)::bigint),
        CASE WHEN TG_OP != 'INSERT' THEN row_to_json(OLD)::jsonb END,
        CASE WHEN TG_OP != 'DELETE' THEN row_to_json(NEW)::jsonb END,
        current_setting('app.current_user', true) -- set by application
    );
    RETURN COALESCE(NEW, OLD);
END;
$$;

-- Attach to any table:
CREATE TRIGGER trg_orders_audit
AFTER INSERT OR UPDATE OR DELETE ON orders
FOR EACH ROW EXECUTE FUNCTION audit_trigger_fn();

```

Upsert helper

```

CREATE OR REPLACE FUNCTION upsert_product(
    p_id          INT,
    p_name        TEXT,
    p_price       NUMERIC,
    p_category    TEXT
)
RETURNS TABLE (product_id INT, action TEXT)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    INSERT INTO products (id, name, price, category)
    VALUES (p_id, p_name, p_price, p_category)
    ON CONFLICT (id) DO UPDATE SET
        name      = EXCLUDED.name,
        price     = EXCLUDED.price,
        category  = EXCLUDED.category,
        updated_at = now();
RETURNING
    id,
    CASE xmax WHEN 0 THEN 'inserted' ELSE 'updated' END;
END;
$$;

```

Production Monitoring Queries

```

-- Find all user-defined functions and procedures
SELECT
    routine_name,
    routine_type,
    data_type AS return_type,
    routine_definition IS NOT NULL AS has_body
FROM information_schema.routines
WHERE routine_schema = 'public'
    AND routine_type IN ('FUNCTION', 'PROCEDURE')
ORDER BY routine_type, routine_name;

-- Function details from catalog
SELECT
    p.proname AS function_name,
    pg_get_function_arguments(p.oid) AS arguments,
    pg_get_function_result(p.oid) AS return_type,
    p.prosecdef AS security_definer,
    p.provolatile AS volatility, -- i=immutable, s=stable, v=volatile
    p.proisstrict AS strict,
    l.lanname AS language
FROM pg_proc p
JOIN pg_namespace n ON n.oid = p.pronamespace
JOIN pg_language l ON l.oid = p.prolang
WHERE n.nspname = 'public'

```

```

    AND p.prokind IN ('f', 'p') -- f=function, p=procedure
ORDER BY p.proname;

-- Functions consuming most time
SELECT
    query,
    calls,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec,
    ROUND(mean_exec_time::numeric, 2) AS avg_ms
FROM pg_stat_statements
WHERE query ILIKE '%SELECT % FROM %(' -- function calls in SQL
    OR query ILIKE '%CALL %'
ORDER BY total_exec_time DESC
LIMIT 10;

-- Functions with invalid body (dependency check)
SELECT routine_name
FROM information_schema.routines
WHERE routine_schema = 'public'
    AND routine_type = 'FUNCTION';
-- Then run: SELECT function_name(); to test each

```

Common Mistakes

1. **Missing OR REPLACE** — Without `CREATE OR REPLACE`, re-creating an existing function throws an error. Always use `CREATE OR REPLACE FUNCTION`.
2. **Using VOLATILE functions in WHERE clauses** — `SELECT * FROM orders WHERE expensive_function(id) = 'value'` calls the function for every row. Use `STABLE` or `IMMUTABLE` where correct, or pre-compute the value.
3. **Not handling NULL inputs** — PL/pgSQL functions don't automatically return NULL for NULL inputs unless `STRICT` is specified. Add explicit NULL checks.
4. **SECURITY DEFINER without SET search_path** — A SECURITY DEFINER function that omits `SET search_path = ...` is vulnerable to search_path injection. Always add `SET search_path = public, pg_catalog`.
5. **Large result sets with RETURNS TABLE** — Returning millions of rows from a function defeats the purpose. Add `LIMIT` / pagination parameters.
6. **Not using RETURN QUERY** — In functions returning `TABLE` or `SETOF`, use `RETURN QUERY SELECT ...` not a plain `SELECT`. Plain `SELECT` in plpgsql doesn't automatically return rows.
7. **Functions in CHECK constraints calling non-immutable functions** — Check constraints run on every INSERT/UPDATE. Functions in CHECK must be `IMMUTABLE`.

Best Practices

1. **Default to SQL functions** for simple operations — they're inlined by the planner, faster, and simpler.
2. **Use STRICT** for functions that should return NULL when any input is NULL.

3. Add `LANGUAGE sql IMMUTABLE` for pure computations (no DB access).
 4. **Parameterize everything** — Never concatenate user input into SQL strings in PL/pgSQL (SQL injection).
 5. **Test volatility** — Wrong volatility (e.g., `STABLE` for a function that modifies data) causes subtle correctness bugs.
 6. **Commit in procedures, not functions** — Use procedures for multi-step operations needing intermediate commits.
-

Performance Considerations

```
-- Function overhead measurement
EXPLAIN (ANALYZE, TIMING)
SELECT calculate_discount(total, 'gold') FROM orders LIMIT 1000;
-- Volatile function: called 1000 times
-- Immutable function with same inputs: called once, result cached per plan node

-- Inline SQL functions (verify with EXPLAIN)
CREATE FUNCTION double(x INT) RETURNS INT LANGUAGE sql IMMUTABLE AS $$ SELECT x * 2
$$;
EXPLAIN SELECT double(id) FROM orders; -- shows: "Output: (id * 2)" — inlined!

-- Pre-compute expensive function results
CREATE MATERIALIZED VIEW order_discounts AS
SELECT id, calculate_discount(total, tier) AS discounted_price
FROM orders JOIN customers USING (customer_id);
```

Interview Questions & Answers

Q1: What is the difference between a PostgreSQL function and a procedure?

A: Functions must return a value and cannot use `COMMIT / ROLLBACK` — they always execute within the caller's transaction. They can be used in SQL expressions (`SELECT` , `WHERE` , `ORDER BY`). Procedures (added in PostgreSQL 11) don't return values but can control transactions using `COMMIT / ROLLBACK` — ideal for long-running batch operations that need intermediate commits. Procedures are called with `CALL` , not `SELECT` .

Q2: Explain the three volatility categories and their impact on query planning.

A: `IMMUTABLE` — same inputs always produce the same output, no DB access. The planner can evaluate the function once at plan time and use the cached result. `STABLE` — within a single query execution, same inputs produce the same result. Can read DB. The planner calls it once per query plan, not once per row. `VOLATILE` (default) — can do anything; evaluated for every row. Never inlined or cached. Using the wrong category (e.g., marking as `STABLE` a function that has side effects) causes incorrect results.

Q3: What is `SECURITY DEFINER` and what are its risks?

A: `SECURITY DEFINER` makes the function execute with the privileges of the function's owner, not the caller. Useful for allowing unprivileged users to perform specific privileged operations. Risks: (1) SQL injection in the function body can give the attacker owner-level access, (2) `search_path` manipulation — a malicious schema

can shadow system functions. Mitigation: always add `SET search_path = public, pg_catalog` and parameterize all SQL.

Q4: How do you return multiple rows from a PL/pgSQL function?

A: Use `RETURNS TABLE(col1 type, col2 type, ...)` with `RETURN QUERY SELECT ...` inside the body. Or use `RETURNS SETOF existing_type` with `RETURN NEXT` or `RETURN QUERY`. For complex types, return `SETOF RECORD` with `OUT` parameters.

Q5: What is the difference between `RETURN NEXT` and `RETURN QUERY` ?

A: `RETURN NEXT value` appends a single row to the result set and continues execution. `RETURN QUERY SELECT ...` appends all rows from the query to the result set at once. `RETURN QUERY` is more efficient for large result sets; `RETURN NEXT` is useful when you're computing each row individually in a loop.

Q6: How do you pass an application-level "current user" context to a trigger for audit logging?

A: Use `SET LOCAL app.current_user = 'username'` at the application session level (after connecting). In the trigger function, retrieve it with `current_setting('app.current_user', true)` — the `true` parameter means return NULL if not set rather than raising an error. This is the recommended pattern for application-level audit attribution.

Q7: Why should you avoid concatenating user input in PL/pgSQL dynamic SQL?

A: String concatenation in dynamic SQL is vulnerable to SQL injection. Use `EXECUTE ... USING` with parameters: `EXECUTE 'SELECT * FROM ' || quote_ident(table_name) || ' WHERE id = $1' USING user_id`. Use `format('%I', identifier)` for identifiers and `format('%L', literal)` for literal values.

Q8: When would you use a DO block vs a stored function?

A: `DO` blocks are for one-time, non-reusable operations — like data migrations, one-off backfills, or schema changes. They're anonymous and not stored. Functions are stored, reusable, can accept parameters, return values, and be called from application code. Use `DO` for ops scripts; use functions for reusable business logic.

Exercises and Solutions

Exercise 1

Write a function `calculate_order_total(order_id BIGINT)` that returns the sum of `quantity * unit_price` from `order_items`, and a 10% tax applied on top.

Solution:

```
CREATE OR REPLACE FUNCTION calculate_order_total(p_order_id BIGINT)
RETURNS NUMERIC LANGUAGE plpgsql STABLE AS $$
DECLARE
    v_subtotal NUMERIC;
BEGIN
    SELECT SUM(quantity * unit_price)
    INTO v_subtotal
    FROM order_items
    WHERE order_id = p_order_id;
```

```

    IF NOT FOUND OR v_subtotal IS NULL THEN
        RAISE EXCEPTION 'Order % not found or has no items', p_order_id;
    END IF;

    RETURN ROUND(v_subtotal * 1.10, 2); -- add 10% tax
END;
$$;

SELECT calculate_order_total(1);

```

Exercise 2

Write a procedure `archive_old_orders(p_before DATE)` that moves orders older than `p_before` to an `orders_archive` table in batches of 1000, committing after each batch.

Solution:

```

CREATE TABLE IF NOT EXISTS orders_archive (LIKE orders INCLUDING ALL);

CREATE OR REPLACE PROCEDURE archive_old_orders(p_before DATE)
LANGUAGE plpgsql AS $$
DECLARE
    v_moved INT;
    v_total INT := 0;
BEGIN
    LOOP
        WITH moved AS (
            DELETE FROM orders
            WHERE id IN (
                SELECT id FROM orders
                WHERE created_at < p_before
                LIMIT 1000
            )
            RETURNING *
        )
        INSERT INTO orders_archive SELECT * FROM moved;

        GET DIAGNOSTICS v_moved = ROW_COUNT;
        v_total := v_total + v_moved;
        COMMIT;
        EXIT WHEN v_moved = 0;
        RAISE NOTICE 'Archived % rows (total: %)', v_moved, v_total;
    END LOOP;
    RAISE NOTICE 'Archive complete. Total archived: %', v_total;
END;
$$;

CALL archive_old_orders('2023-01-01');

```

Cross-References

- **10_triggers_and_rules.md** — Trigger functions built on PL/pgSQL
- **08_listen_notify.md** — pg_notify() in trigger functions
- **04_materialized_views.md** — Refresh functions
- **09_transactions_concurrency** — Transaction control in procedures
- **PostgreSQL Docs** — <https://www.postgresql.org/docs/current/plpgsql.html>